Algorithms for Our Rig: Jetson AGX Orin 64 GB + Livox Mid-360 + ZED 2i

2026-10-18. Builds on 03-algorithms-software, narrowed to the hardware we actually have. ***[verify]*** *= check before relying on it.*

1. What the hardware gives us (and what it doesn't)

Sensor	Gives us	Limitations that shape our choices
Livox Mid-360	360°×59° FoV, ~200k pts/s, 0.1-40 m, ~2 cm range noise, **built-in 6-axis IMU**, Ethernet, supports **PTP** time sync	Vertical FoV is only −7° to +52°, so it sees little floor near the robot; consider tilting the mount. Non-repetitive scan pattern, so a single frame is sparse; that is fine for LIO and needs accumulation for mapping.
ZED 2i	Stereo 2×1080p/2K, 12 cm baseline, 110° FoV, depth to ~20 m, factory-calibrated, IMU + barometer + magnetometer, ZED SDK on Jetson (neural depth, tracking, object and body detection)	**Rolling shutter** gives motion blur and skew when moving fast. **USB with no hardware trigger**, so LiDAR-camera sync is software-only. Forward-facing only, with no 360° view.
AGX Orin 64 GB	275 TOPS, 12-core ARM, 64 GB unified memory, JetPack 6 (Ubuntu 22.04)	**ROS 2 Humble**, not Jazzy (Jazzy needs Ubuntu 24.04). That corrects my earlier stack note. Heavy reconstruction and large training jobs are better on a desktop GPU.

What's missing: RTK GNSS, so the twin will be in a **site-local frame**. A ~$200 UM980/F9P board can be added later for georeferencing. We also have no 360° camera, so we need more passes for full visual coverage.

2. Algorithm choices, layer by layer

Layer 1: Odometry and SLAM (real-time on the Orin)

Role	Primary pick	Alternatives	Why
LiDAR-inertial odometry	**FAST-LIO2** (official `ROS2` branch, ships a `mid360.yaml` config)	Point-LIO (handheld or jerky motion), KISS-ICP (sanity baseline)	Standard algorithm for the Mid-360. Uses about 1-2 CPU cores on the Orin. Uses the Mid-360's internal IMU, whose extrinsic is already known.

Role	Primary pick	Alternatives	Why
LiDAR + camera odometry (coloured map)	**FAST-LIVO2** with ZED left image	R3LIVE (older)	Coloured point cloud plus robustness in corridors and on blank slabs where geometry is degenerate. **Risk:** it expects hardware-synced cameras. With the ZED over USB we have to estimate the time offset, and the official code is ROS 1, with ROS 2 community ports **[verify]**.
Loop closure + global map + multi-session	**GLIM** (+ `glim_ext`), CUDA-accelerated, ROS 2, Livox-ready	FAST-LIO + Scan Context pose graph (FAST-LIO-SAM style); LT-mapper	Fixes drift and merges captures from different days into one frame. That alignment is the foundation for change detection. Run it offline on the Orin or a desktop. Jetson CUDA build **[verify]**.
Visual relocalization (optional)	ZED SDK positional tracking / area memory; hloc offline	ORB-SLAM3 (GPL)	Backup when LiDAR degenerates, and for relocating the robot on an existing map.

Without RTK: anchor each site to 3-5 fixed markers (ArUco/AprilTag boards, or surveyed nails measured by tape or total station) so all captures share one site frame.

Layer 2: Geometry (dense map, mesh)

Role	Pick	Where it runs
Live occupancy, TSDF and mesh for navigation and preview	**nvblox** (Isaac ROS), fusing ZED depth and/or Mid-360 points on the GPU	Orin, real time
Quick baseline	ZED SDK spatial mapping (mesh or fused cloud)	Orin
Final point cloud	GLIM map → **PDAL** filtering/decimation → **COPC/LAZ**	Orin or desktop
Clean mesh	VDBFusion / Open3D TSDF, or screened Poisson	Desktop

Layer 3: Photoreal visual twin (Gaussian splatting)

- **Inputs:**
  - ZED 2i left (and right) images at 2K/15 fps or 1080p/30 fps, with fixed exposure.
  - Poses from FAST-LIVO2/GLIM, transformed by the LiDAR-camera extrinsic.
  - Mid-360 coloured points as the initial Gaussians.
- **Training:**
  - **gsplat** with LiDAR depth supervision, which fixes scale and floaters on blank concrete.
  - **2DGS / PGSR** when we need measurable surfaces.
  - Skip COLMAP pose estimation, since we already have metric poses. Optionally refine poses inside GS training.
- **Where:** the Orin's 64 GB unified memory can train **small scenes** (a room or a floor section) slowly. Use a desktop RTX GPU or cloud for full-site scenes.
- **Capture rules:** drive slowly (rolling shutter), make overlapping passes in both directions, and avoid pointing straight into the sun.
- **Dynamic objects:** mask people and machines with YOLO/SAM masks before training.

Layer 4: Semantics (what is where)

Role	Pick	Where
People, PPE, equipment detection	**YOLO11 / RT-DETR / RF-DETR → TensorRT**, fine-tuned on **SODA + MOCS + ACID**	Orin, real time (20-30+ fps)
People tracking (quick win)	ZED SDK body tracking / object detection	Orin
Tracking	ByteTrack / BoT-SORT	Orin
Open-vocabulary labels ("rebar", "formwork", "duct")	Grounding DINO + **SAM 2/3**	Desktop or offline on the Orin
Lifting 2D labels to 3D	Project Mid-360 points into the labelled image using the extrinsic, then vote across frames	Orin or desktop
3D segmentation	Pointcept **PTv3 / Sonata** (fine-tuned later on our own labels)	Desktop

License note: Ultralytics YOLO is AGPL. RT-DETR and RF-DETR (Apache) are safer for a product.

Layer 5: Time / 4D (what changed)

1. **Cleaning:** remove dynamic objects with **DUFOMap** / **ERASOR** / **MapMOS**, and keep what was removed as the "activity" layer.
2. **Alignment between captures:** GLIM multi-session, or **TEASER++** (coarse) + **small_gicp** (fine) against the reference capture and the site markers.
3. **Comparison:**
   - **M3C2** (py4dgeo) for significance-aware differences.
   - Voxel occupancy diff for added and removed volume.
   - DSM differencing for earthworks.
4. **Later:** scan-to-BIM with **IfcOpenShell**, giving per-element C2M deviation and percent complete.

3. Calibration and sync (do this first; everything depends on it)

Task	Tool	Note
ZED intrinsics + stereo	Factory calibration (ZED SDK)	Good enough
Mid-360 LiDAR ↔ IMU	Known factory offset (already in the FAST-LIO `mid360.yaml`)	
Mid-360 ↔ ZED extrinsic	**`direct_visual_lidar_calibration`** (koide3, targetless, ROS 2, supports Livox)	Alternative: `livox_camera_calib` (hku-mars). Accumulate a few seconds of static Mid-360 data for a dense cloud.
ZED IMU ↔ camera (only if we use the ZED IMU)	Factory calibration / Kalibr	Prefer the Mid-360 IMU as the main IMU.
Time sync	Mid-360 via **PTP** (`ptp4l` on the Orin as master); ZED uses host timestamps	Estimate the camera-LiDAR time offset (FAST-LIVO2 has a parameter for it). Validate by waving a target and checking reprojection.
Mounting	One rigid bracket for both sensors	Any flex ruins the calibration. Re-check calibration after transport.

4. Software stack on the Orin

- JetPack 6.x, **ROS 2 Humble**, `livox_ros_driver2` (Livox-SDK2), `zed-ros2-wrapper` (ZED SDK for L4T)
- FAST-LIO2 (ROS 2), GLIM, Isaac ROS nvblox, TensorRT
- Recording with `ros2 bag` in **MCAP** format, viewed in Foxglove or Rerun. Data rate is about 0.5-1 GB/min with compressed images, so use an NVMe SSD.

Compute budget on the Orin (rough estimate)

Process	Load
Livox driver + FAST-LIO2	~1-2 CPU cores
ZED SDK depth (NEURAL)	~30-40% GPU
YOLO TensorRT (1 stream)	~15-25% GPU
nvblox	~10-20% GPU
Recording (MCAP, compressed)	1 core + disk

This fits comfortably. GLIM global optimisation and GS training run after the capture.

5. Expected accuracy

- **FAST-LIO2 alone:** ~0.1-1% drift of distance travelled.
- **With loop closure (GLIM) + markers: ~2-5 cm** across a site, ~1-2 cm locally.
- **What that supports:** progress tracking (LOA10-20), visual twin, navigation maps, change detection of objects larger than ~5 cm. It does **not** support QA-grade deviation (5-15 mm), which needs a borrowed TLS to validate.

6. Build order

1. **Bring-up:** drivers for both sensors in ROS 2 Humble on the Orin; record MCAP.
2. **Calibrate:** Mid-360↔ZED extrinsic + PTP sync; verify by overlaying LiDAR points on the ZED images.
3. **FAST-LIO2 live** → walk or drive a test area → check drift with a loop.
4. **GLIM offline** → globally consistent map → COPC → view in Potree/CloudCompare.
5. **Second capture on another day** → multi-session alignment → M3C2 change map. **This is the first real "4D twin" demo.**
6. **Coloured map + Gaussian splat** from the same data (FAST-LIVO2 poses → gsplat).
7. **Semantics:** YOLO/RT-DETR on the Orin + 3D lifting; people and equipment layer.
8. Later: RTK, BIM registration, autonomous routes, live sensors.